

CSA0619-DESIGN ANALYSIS OF ALGORITHM

CO5 AT2 ANSWERS

BY ANTONY MAGRIN J(192572103)

Q3. Graph Coloring Problem — Backtracking Solution

1. Problem Definition

Input:

- graph — a graph $G=(V,E)$ on n vertices (given as an adjacency matrix/list)
- M — the number of colors available (labeled $1..M$)

Output:

- An array $color[0..n-1]$ assigning each vertex a color in $\{1..M\}$ such that for every edge $(u,v) \in E$, $color[u] \neq color[v]$, or a report that **no such coloring exists**.

Constraint: for every edge, its two endpoints must differ in color — this is the only constraint, but it must hold simultaneously for all edges.

2. Pseudocode

ALGORITHM GraphColoring(graph, M)

INPUT : graph -> adjacency matrix $graph[n][n]$, $graph[i][j] = 1$ if
edge (i,j) exists

M -> number of colors available

OUTPUT: $color[0..n-1]$ such that no two adjacent vertices share a
color,

or "NO SOLUTION" if impossible

1 $n \leftarrow$ number of vertices in graph

```

2  color[0..n-1] <- 0 // 0 means "uncolored"
3
4  FUNCTION isSafe(vertex, c, color): // validation
function
5      FOR each u in 0..n-1:
6          IF graph[vertex][u] == 1 AND color[u] == c:
7              RETURN false // adjacent vertex
already has color c
8      RETURN true
9
10 FUNCTION solve(vertexIndex, color): // recursive
backtracking
11     IF vertexIndex == n: // BASE CASE: all
vertices colored
12         RETURN true // a complete
valid coloring found
13
14     FOR c <- 1 TO M: // try each
color for this vertex
15         IF isSafe(vertexIndex, c, color): // constraint
check
16             color[vertexIndex] <- c // tentative
assignment (choose)
17
18             IF solve(vertexIndex + 1, color): // recurse to
next vertex
19                 RETURN true // propagate
success upward
20
21             color[vertexIndex] <- 0 // undo
assignment (backtrack)
22     RETURN false // no color
worked at this vertex

```

```

23
24  MAIN:
25      IF solve(0, color) == true:
26          PRINT color[]                // valid M-coloring
found
27      ELSE:
28          PRINT "NO SOLUTION"          // graph is not M-
colorable
-----

```

3. Design Notes — Recursion, Loops, and Validation

- **Recursion (solve)** advances vertex-by-vertex ($\text{vertexIndex} \rightarrow \text{vertexIndex}+1$); the base case $\text{vertexIndex} == n$ is reached only when every vertex holds a legal color — that is a complete solution.
- **Loop (FOR c <- 1 TO M)** systematically tries every color at the current vertex, so no candidate is skipped.
- **Validation (isSafe)** is the constraint-check/pruning function: it only inspects already-colored neighbors, rejecting a color the instant it conflicts with any of them — this stops the recursion from ever descending into a doomed branch.
- **Backtracking step** ($\text{color}[\text{vertexIndex}] <- 0$ after a failed recursive call) undoes the tentative choice so the next candidate color (or the parent call) can try a fresh possibility — this is what makes the search exhaustive over the valid space without permanently committing to a bad choice.

4. Completeness & Correctness

- **Completeness:** because every color $1..M$ is tried at every vertex (loop, line 14) and every failure triggers backtracking (line 21) rather than giving up, the algorithm implicitly explores the entire relevant search tree — it returns true if any valid coloring exists, and only returns false after exhausting all possibilities.
- **Correctness:** isSafe guarantees every accepted assignment respects all edges seen so far, and the base case only fires once all n vertices are assigned — so any returned solution is fully valid by construction.

- I verified this empirically against **brute-force enumeration** (`itertools.product`) on a small graph — both methods agreed on solvability for $M=2$ (unsolvable) and $M=3$ (solvable), confirming completeness.

5. Execution Results

```
Australia      map      (7      regions,      adjacency-based):
M=2              ->              NO              SOLUTION
M=3  ->  {WA:1,  NT:2,  SA:3,  Q:1,  NSW:2,  V:1,  T:1}    ✓  valid
M=4  ->  valid (same as M=3 solution, extra color unused)
```

```
Triangle              graph              K3:
M=2  ->  NO  SOLUTION      (confirms K3 needs ≥3 colors)
M=3    ->    [1,    2,    3]              ✓    valid
```

```
Completeness  check  vs  brute  force  (K4-minus-edge,  n=4):
M=2:    backtracking=False,    brute-force=False    →    match
M=3:    backtracking=True,    brute-force=True    →    match, solution
[1,2,3,1]
```

Complexity: worst case $O(M^n)$ (bounded search tree), but the `isSafe` pruning cuts off infeasible branches immediately — in practice far fewer than M^n nodes are visited, especially on sparse graphs.